

CaterPro SaaS System

Comprehensive Architectural & Technical Documentation

This document provides a comprehensive blueprint, implementation strategy, and technical requirement specification for the CaterPro Multi-tenant SaaS Catering Management System.

1. System Overview

CaterPro is a dual-application architecture built with Laravel (versions 11/12) and React. It operates through a Central Application (Landlord) for platform management and an isolated Tenant Application for business operations.

Key Technological Stack:

- Backend: Laravel 11/12
- Frontend: React 19, Vite, TailwindCSS v4
- Tenancy Engine: stancl/tenancy v3
- Authentication: Laravel Sanctum
- Roles & Permissions: spatie/laravel-permission

2. Key Architectural Components

Tenancy Strategy

The system utilizes multi-database isolation where every tenant is provisioned with a dedicated MySQL/SQLite database. Tenant identification is performed via subdomain resolution. Storage, cache, and filesystem access are also strictly isolated per tenant using tenancy bootstrappers.

Central App (Landlord)

Handles platform-level operations, including:

- Tenant Provisioning (TenantProvisioningService)
- Subscription Tiering (Free, Starter, Business, Enterprise)
- Revenue Analytics & Global System Health

Tenant App (Business Context)

Scoped by tenancy middleware, the Tenant App includes core catering modules:

- Event Management (lifecycle, Kanban, calendar view)
- Client Portal & CRM
- Staff Assignment & Scheduling
- Package Engineering (Menu Builder)
- Payments & Invoicing

3. State Management & Frontend Architecture

React Context (TenantProvider)

The core of the frontend state is managed by the TenantProvider. It handles runtime checks for tenant capabilities, active subscription features, and authenticated user roles. Recent refactoring shifted from unstable `useQuery`-based auth logic to a deterministic `useState/useEffect` pattern, eliminating redirect loops between login and workspace components.

Dynamic UI & Branding

The UI utilizes CSS Custom Properties injected by the TenantProvider to allow for white-labeling per tenant. Tenants on higher-tier plans can customize primary colors and logos.

4. Access Control & Subscription Tiers

Roles and Permissions (RBAC)

Stored within the tenant database, the system defines four core roles:

- Admin: Full access to all modules, including settings and billing.
- Manager: Access to events, clients, packages, staff, and analytics.
- Staff: Access to dashboard, events, and clients.
- Cashier: Access to dashboard, payments, and events.

Subscription Plans

- Free: Event Management (Limit 10 active events/month, 3 users)
- Starter: Adds Client Portal, Staff Assignment
- Business: Adds Advanced Analytics, Branding Controls
- Enterprise: Unlimited users/events, all features

5. Security & Operational Preferences

Feature Gating

Features are enforced via the `EnsureTenantFeatureEnabled` middleware on the backend, and the `RequireTenantModule` component on the frontend, ensuring strict compliance with subscription plans.

Active Tenant Check

The `EnsureTenantIsActive` middleware blocks access to suspended tenant workspaces.

Data Isolation

Strict separation ensures tenant queries are scoped to the current database instance, completely preventing cross-tenant data leakage.

6. Core Workflows & Services

Tenant Provisioning Flow

The `TenantProvisioningService` encapsulates the full lifecycle, including domain creation, database migration, and initial RBAC seeding (via `TenantRbacSeeder`).

Event Quotas

The Event status system is tied to financial triggers managed by the `EventQuotaService`. Active event counts are strictly monitored based on plan limits.